

DMA Simülatörü ve Bus Arbitration (Veri Yolu Tahkimi) Proje Raporu

1. Proje Özeti ve Amacı

Bu projenin amacı, Bilgisayar Mimarisi dersi kapsamında **Doğrudan Bellek Erişimi (DMA - Direct Memory Access)** ve **Veri Yolu Tahkimi (Bus Arbitration)** kavramlarının yazılımsal bir simülasyonunu gerçekleştirmektir. Simülasyon, fiziksel bir donanım gerektirmeden, işlemci (CPU), bellek (RAM) ve DMA kontrolcüsünün ortak bir veri yolunu (Bus) nasıl paylaştığını Nesne Yönelimli Programlama (OOP) prensipleri kullanılarak Java dilinde modellenmiştir.

2. Temel Kavramlar

- DMA (Direct Memory Access):** Giriş/Çıkış (I/O) cihazlarının, işlemciyi (CPU) meşgul etmeden doğrudan belleğe (RAM) veri yazmasını veya bellekten veri okumasını sağlayan donanım bileşenidir. İşlemci sadece transferin başında DMA'yı yapılandırır ve transfer bittiğinde DMA işlemciye bir kesme (Interrupt) gönderir.
- Bus Arbitration (Veri Yolu Tahkimi):** Aynı anda birden fazla cihazın (örneğin CPU ve DMA) ortak veri yolunu (System Bus) kullanmak istemesi durumunda yaşanacak çakışmaları (collision) önleme mekanizmasıdır. Bu projede, hakem (Arbiter) rolünü üstlenen bir yapı ile önceliklendirme sağlanmıştır.

3. Sistem Mimarisi ve Sınıf (Class) Tasarımı

Proje, donanım bileşenlerini temsil eden temel sınıflara ayrılmıştır. Bu modüler yapı sayesinde her bir bileşen kendi görevini yerine getirirken, sistem saati (Clock) eşliğinde senkronize bir şekilde çalışırlar.

Sınıf (Class) Adı	Donanım Karşılığı	Görevi ve Açıklaması
RAM	Rastgele Erişimli Bellek	Sistem belleğini temsil eden basit bir tamsayı (int) dizisidir. Kendisine verilen adrese veri yazar veya verilen adresten veri okur.
BusArbiter	Veri Yolu Hakemi	CPU ve DMA'nın veri yoluna erişim taleplerini değerlendirir. Veri yolu doluysa (örneğin DMA kullanıyorsa), CPU'nun beklemesini (Stall) sağlar. "Daisy Chaining" veya "Round-Robin" benzeri bir önceliklendirme mantığının temelini oluşturur.
DMA	DMA Kontrolcüsü	CPU'dan aldığı emir (başlangıç

Sınıf (Class) Adı	Donanım Karşılığı	Görevi ve Açıklaması
		adresi ve miktar) doğrultusunda, Arbiter'den izin alabildiği "clock cycle"larda (saat vuruşlarında) RAM'e veri transferini gerçekleştirir. Transfer bitince veri yolunu bırakır (Release Bus).
CPU	Merkezi İşlem Birimi	Simülasyonda DMA'yı başlatan ve kendi işleyişi sırasında veri yoluna erişmeye çalışan birimdir. Veri yolu DMA tarafından işgal edildiğinde beklemek zorundadır.
Main	Sistem Saati (Clock Generator)	Simülasyonun ana döngüsünü (while/for loop) barındırır. Bilgisayarın osilatörünü taklit ederek her döngü adımında (tick) sistemdeki tüm cihazların durumunu günceller.

4. Simülasyon Senaryosu (Örnek Çalışma)

Hazırlanan kod iskeleti üzerinden işletilen örnek senaryo şu şekildedir:

1. **Başlangıç:** Sistem bileşenleri (RAM, Arbiter, DMA) başlatılır.
2. **Emir:** CPU, DMA kontrolcüsüne 0. adresten başlayarak 3 birimlik veri transfer etmesi komutunu gönderir.
3. **Clock 1 (Saat Vuruşu 1):** DMA, Arbiter'den (Hakem) veri yolunu ister ve alır. RAM'in 0. adresine ilk veriyi yazar.
4. **Clock 2:** DMA veri transferine devam ederken (RAM adres 1'e yazarken), CPU kendi işi için veri yolunu kullanmak ister. Arbiter, yolun DMA'da olduğunu belirterek CPU'ya "bekle" (Stall) der. Bu durum *Bus Arbitration*'in çalıştığını gösterir.
5. **Clock 3:** DMA son verisini RAM adres 2'ye yazar, transferi tamamlar. CPU'ya "İşlem Tamam" (Interrupt) sinyalini simüle ederek veri yolunu serbest bırakır (Release).
6. **Sonuç:** RAM'in güncel durumu ekrana (konsola) basılır ve veri bütünlüğü doğrulanır.

5. Kurulum ve Çalıştırma Talimatları

Bu proje Java programlama dili ile geliştirilmiştir. Aşağıdaki adımları izleyerek projeyi çalıştırabilirsiniz:

1. **Gereksinimler:** Bilgisayarınızda Java Development Kit (JDK 8 veya üzeri) ve IntelliJ IDEA (veya benzeri bir IDE) kurulu olmalıdır.
2. **Proje Oluşturma:** IntelliJ IDEA'da yeni bir Java projesi oluşturun (Örnek: DMASimulator).
3. **Kodu Ekleyin:** Hazırlanan Main.java kod dosyasını projenin src klasörü altına yapıştırın. (Kod, önceki aşamada sağlanan konsol tabanlı simülasyon iskeletidir).
4. **Çalıştırma:** Main metodunu barındıran dosyayı seçip sağ tıklayarak **Run 'Main.main()'**

seeneęine tıklayın.

5. **Çıktı Kontrolü:** Konsol ekranında saat vuruşlarına (clock cycle) göre cihazların durumlarını ve RAM'in son durumunu inceleyin.

6. Sonuç ve Gelecek Geliştirmeler

Bu simülasyon, donanım derslerinde teorik olarak işlenen veri yolu paylaşımlarının ve DMA'nın I/O işlemlerindeki yükü CPU'dan nasıl aldığıının pratik bir kanıtını sunmaktadır. Gelecek geliştirmeler için projeye Tkinter (Python) veya Java Swing/JavaFX kullanılarak görsel bir arayüz (GUI) eklenebilir. Böylece hangi donanımın veri yolunu kullandığı anlık olarak ışıklı göstergelerle sunulabilir.